

評価値（評価コスト $f(x) = g(x) + h(x)$ ）を用いた探索

準備

- $g(v)$ は $s \sim v$ のコスト (の暫定値), $h(v)$ は $v \sim g$ のコスト (の推定値).
- $[\dots]$ はノード(状態)のリストを表す.
- 変数 OL, CL, P_A はノード(状態)のリスト, 変数 A は 1 つのノード(状態)を表す.
- s は初期状態を表す.
- $+$ はリスト連結の演算子とする.
- $\text{pop}(l)$ は, リスト l の先頭(左端)から要素を一つとりだす関数とする.
- $\text{next}(x)$ は, x の隣接ノード(次の状態)のリストを返す関数とする. その際, 各ノードに x へのリンクを持たせる. (探索木や経路を生成するのに必要)
- $\text{sort}(l)$ は, リスト l の要素を, その評価コスト ($f(x) = g(x) + h(x)$) で昇順に並べ替えたリストを返す関数とする.
- $\text{uniq}(l)$ は, リスト l の重複を(最左の要素のみを残すことによって)取り除いたリストを返す関数とする.

探索手続き（来し方・行く末コストを用いた探索）

- 1 OL $\leftarrow$ [s]; CL $\leftarrow$ [];
 - 2 もし OL が空なら, 失敗として終了;
 - 3 A $\leftarrow$ pop(OL);
 - 4 もし A が目標状態なら, 成功として終了;
 - 5 $P_A \leftarrow$ next(A);
 - 6 CL $\leftarrow$ CL + [A];
 - 7 P_A の要素の各々(v)について, 以下を行う.
 - 7.1 もし v が CL に含まれ, かつ, P_A 中の v の $f(v)$ が CL 中の v の $f(v)$ 以上なら,
 P_A から v を取り除く; // 探索済みの状態を除外
 - 7.2 もし v が CL に含まれ, かつ, P_A 中の v の $f(v)$ が CL 中の v の $f(v)$ 未満なら,
 CL から v を取り除く; // $s \sim v$ のコスト $g(v)$ が更新された場合,
 // v を CL から OL に戻す
 - 8 OL $\leftarrow$ sort(OL + P_A); // 評価値(評価コスト)の昇順に並べ替える
 - 9 OL $\leftarrow$ uniq(OL); // 重複は, 評価値(評価コスト)の悪いものを除去
 - 10 goto 2
- ※ P_A 中の v の $f(v)$ が CL 中の v の $f(v)$ より小さい場合, $h(v)$ は変化しないので, $g(v)$ が更新されたことになる.
- ※ $g(v)$ が更新された場合, 先に v を経由して探索された状態を再び探索し直す必要が生じるため, 7.2 のように, v を再び OL に戻す必要がある.

分枝限定探索（来し方コスト $g(x)$ のみを用いた探索, $h(x) \equiv 0$ ）

- 1 OL $\leftarrow$ [s]; CL $\leftarrow$ [];
 - 2 もし OL が空なら, 失敗として終了;
 - 3 A $\leftarrow$ pop(OL);
 - 4 もし A が目標状態なら, 成功として終了;
 - 5 $P_A \leftarrow$ next(A);
 - 6 CL $\leftarrow$ CL + [A];
 - 7 P_A の要素の各々(v)について, 以下を行う.
 - 7.1 もし v が CL に含まれるなら,

P_A から v を取り除く; // 探索済みの状態を除外
 - 8 OL $\leftarrow$ sort(OL + P_A); // 評価値(評価コスト)の昇順に並べ替える
 - 9 OL $\leftarrow$ uniq(OL); // 重複は, 評価値(評価コスト)の悪いものを除去
 - 10 goto 2
- ※ 7.1 で v が CL に含まれている場合, P_A 中の v の評価値 $f(v)$ ($= g(v)$) は常に CL 中の v の評価値以上なので, 上の 7.2 のように, v を CL から OL に戻す必要はない.
- ※ 8 の並べ替えを除けば, グラフの(評価コストを用いない)探索に類似している.

最良優先探索（行く末コスト $h(x)$ のみを用いた探索, $g(x) \equiv 0$ ）

- 1 OL $\leftarrow$ [s]; CL $\leftarrow$ [];
 - 2 もし OL が空なら, 失敗として終了;
 - 3 A $\leftarrow$ pop(OL);
 - 4 もし A が目標状態なら, 成功として終了;
 - 5 $P_A \leftarrow$ next(A);
 - 6 CL $\leftarrow$ CL + [A];
 - 7 P_A の要素の各々(v)について, 以下を行う.
 - 7.1 もし v が CL または OL に含まれるなら,

P_A から v を取り除く; // 探索済みの状態, および重複を除外
 - 8 OL $\leftarrow$ sort(OL + P_A); // 評価値(評価コスト)の昇順に並べ替える
 - 9 goto 2
- ※ 任意の v の評価値 $f(v)$ ($= h(v)$) は常に変化しないので, 上の 7.2 のように, v を CL から OL に戻す必要はなく, 重複の除去も, どちらを除いてもよい.
- ※ 8 の並べ替えを除けば, グラフの(評価コストを用いない)横型探索に類似している.

最急降下法(山登り法)（行く末コスト $h(x)$ のみ. $g(x) \equiv 0$ ）

- ※ 変数 OL, P_A は, ノード(状態)のリストではなく, 1 つのノード(状態)を表す.
- 1 OL $\leftarrow$ s;
 - 2 A $\leftarrow$ OL;
 - 3 もし next(A) が空なら, A を極小点として終了;
 - 4 $P_A \leftarrow$ pop(sort(next(A)));
 - 5 もし $f(P_A)$ が $f(A)$ 以上なら, A を極小点として終了;
 - 6 OL $\leftarrow$ P_A
 - 7 goto 2